Name: _____

Class: _____

Mr. Rawson • GCSE Computer Science Year 11 2026

Algorithms 3 — Linear and Binary Search — ANSWERS

Finding one item in a list is one of the jobs computers do most: a name in a register, a product in a shop's stock, a word in a dictionary. OCR names two standard ways to do it. In the exam you apply each one to a list of data, and recognise each one from its code.

1 The halving game

I am thinking of a number from 1 to 100. After each guess you hear *too high* or *too low*. The best move is to **guess the middle** of the numbers still possible: then, whatever the answer, half of them are gone.

Guesses made	0	1	2	3	4	5	6
Numbers still possible, at most	100	50	25	12	6	3	1

After six guesses only one number is left, so the 7th guess is always right. You played this game in your holiday Guess the Number program: guessing the middle every time is **binary search**, Section 3.

Now try • The game is now from 1 to **1000**. Keep halving: how many guesses are always enough?

Check • 10 guesses. Halving 1000 goes 500, 250, 125, 62, 31, 15, 7, 3, 1: one number is left after 9 guesses, and the 10th finds it. Ten times as many numbers needs only three more guesses.

2 Linear search

VOCAB **Linear search** (OCR 2.1.3) — checks each item in a list in turn, starting with the first, until it finds the target or reaches the end of the list

The rule. Start at index 0. Compare the item with the target. If they match, stop: it is found. If not, move to the next item. If you go past the last item, the target is not in the list. **The list can be in any order.**

PYTHON (what you type)

```
def linear_search(items, target):
    index = 0
    while index < len(items):
        if items[index] == target:
            return True
        index = index + 1
    return False

scores = [14, 9, 31, 22, 5, 17]
print(linear_search(scores, 22))
```

OCR ERL (what the exam shows)

```
1  function LinearSearch(searchList, searchItem)
2    searchIndex = 0
3    while searchIndex < searchList.length
4      if searchList[searchIndex] == searchItem then
5        return true
6      endif
7      searchIndex = searchIndex + 1
8    endwhile
9    return false
10 endfunction
```

The two are the same algorithm: Python's `len(items)` is ERL's `.length`. The `return true` on line 5 ends the function at once, so the search stops as soon as the target is found.

Worked trace. `linear_search(scores, 9)`, using the Python line numbers.

Line	index	index < len(items)	items[index] == target	Return
2	0			
3		True		
4			False (14 is not 9)	
6	1			
3		True		
4			True (9 is 9)	
5				True

Now try • Trace `linear_search(scores, 31)`.

Check • The function returns True on line 5, after comparing 3 items: 14, 9 and 31. The last value of `index` is 2, the position of 31.

Line	index	index < len(items)	items[index] == target	Return
2	0			
3		True		
4			False	
6	1			
3		True		
4			False	
6	2			
3		True		
4			True	
5				True

Now try • Now search `scores` for **8**, which is not in the list. How many items does linear search compare before it returns False?

Check • 6: every item in the list. A linear search can only say *not found* once it has checked them all.

3 Binary search

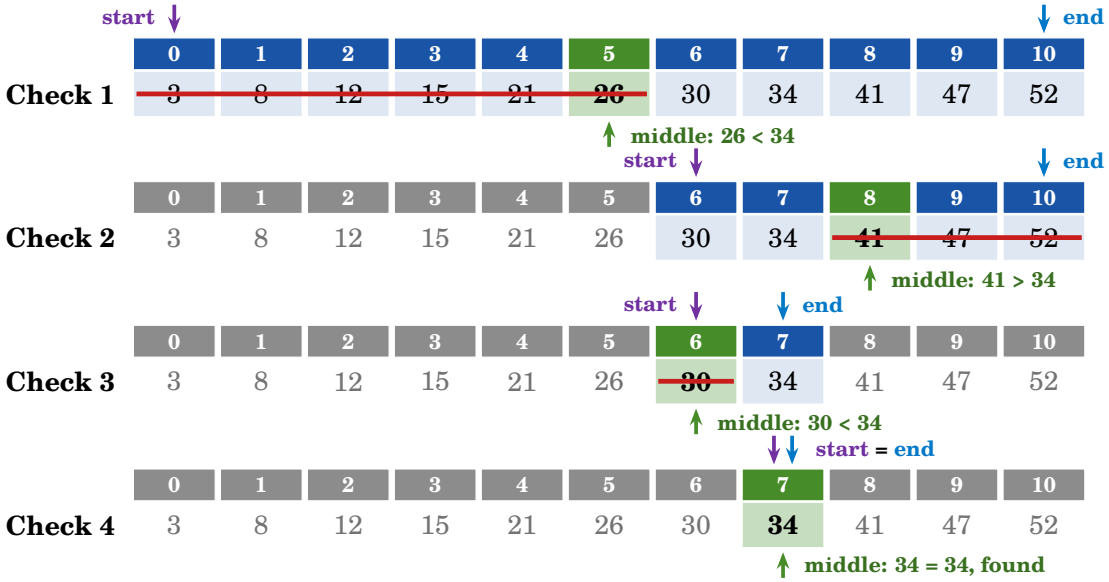
> **VOCAB** **Binary search** (OCR 2.1.3) — checks the middle item of a sorted list, then throws away the half that cannot hold the target, and repeats until the target is found or no items are left

The pre-requisite. The list must be **sorted** (in order). That is what makes it safe to throw half away.

The rule.

1. Find the middle: `middle = (start + end) DIV 2`. `DIV` gives the whole number part, so 6.5 becomes 6.
2. Compare the middle item with the target. If they are equal, it is found: stop.
3. If the middle item is **bigger** than the target, keep the **left** half: `end = middle - 1`. If it is **smaller**, keep the **right** half: `start = middle + 1`.
4. Repeat with the half that is left. When `start` has passed `end`, nothing is left: the target is not in the list.

Worked example. Find **34**. At each check the arrows mark **start** and **end**, the green box is the **middle**, and the red line strikes out the half that is thrown away. Items thrown away at an earlier check are grey.



Step	start	end	middle	Middle item	Compare	So
1	0	10	5	26	26 < 34	keep the right half: start = 6
2	6	10	8	41	41 > 34	keep the left half: end = 7
3	6	7	6	30	30 < 34	keep the right half: start = 7
4	7	7	7	34	34 = 34	**found**, after 4 comparisons

Step 3 has two items left, at indexes 6 and 7. (6 + 7) DIV 2 = 6, so the left one of the two middle items is used. A linear search would have needed 8 comparisons to reach 34.

How to write it in the exam. Name each middle item and the half you discard: *"26 is smaller than 34, so discard 26 and everything to its left. 41 is bigger, so discard 41 and everything to its right. … 34: found."*

Now try • Use the same list to find **12**, then to search for **40**. Fill in a row for each step.

Check • 12 is found at step 2. For 40, the middle items are 26, 41, 30 and 34, then `start` becomes 8 while `end` is 7: start has passed end, so 40 is not in the list.

Target	Step	start	end	middle	Middle item	So
12	1	0	10	5	26	bigger than 12: end = 4
12	2	0	4	2	12	equal: found
40	1	0	10	5	26	smaller than 40: start = 6
40	2	6	10	8	41	bigger than 40: end = 7
40	3	6	7	6	30	smaller than 40: start = 7
40	4	7	7	7	34	smaller than 40: start = 8
40	5	8	7			start has passed end: not found

4 Binary search in ERL: the code to recognise

This is binary search as OCR writes it. You need to recognise it and follow it, not remember it. The Craig 'n' Dave video writes the same algorithm with a `found` flag and `left` and `right` pointers: that version is Fragment B in Section 5.

```
1  function BinarySearch(searchList, searchItem)
2    start = 0
3    end = searchList.length - 1
4    while start <= end
5      midPoint = (start + end) DIV 2
6      if searchList[midPoint] > searchItem then
7        end = midPoint - 1
8      elseif searchList[midPoint] < searchItem then
9        start = midPoint + 1
10     else
11       return true
12     endif
13   endwhile
14   return false
15 endfunction
```

Now try • Match each job to its line number in the ERL above.

Check • Six different line numbers, all between 4 and 14. The two lines that keep a half both change `start` or `end`.

The line that ...	Line
finds the middle position	5
keeps the left half	7
keeps the right half	9
stops the search because the item is found	11
keeps the search going while items are left	4
reports that the item is not in the list	14

5 Linear or binary?

	Linear search	Binary search
Order of the list	any order	must be sorted
Best case	1 comparison: the target is first	1 comparison: the target is in the middle
Worst case, 1000 items	1000 comparisons	10 comparisons
The code	short and simple	longer, more steps to follow
Best used for	short lists, and lists that are not sorted	long, sorted lists

Now try • **Identify the algorithm.** Each fragment searches `names` for `wanted`. Name the algorithm and give the line that tells you.

Check • One of them moves `i` on by 1 each time; the other works out a middle and moves `left` or `right`.

Fragment A

```
1 found = false
2 i = 0
3 while found == false AND i < names.length
4   if names[i] == wanted then
5     found = true
6   endif
7   i = i + 1
8 endwhile
```

Fragment B

```
1  found = false
2  left = 0
3  right = names.length - 1
4  while found == false AND left <= right
5    mid = (left + right) DIV 2
6    if names[mid] == wanted then
7      found = true
8    elseif wanted > names[mid] then
9      left = mid + 1
10   else
11     right = mid - 1
12   endif
13 endwhile
```

	Algorithm	The line that tells you
Fragment A	Linear search	Line 7: `i` moves on by 1 each time.
Fragment B	Binary search	Line 5 works out the middle (and lines 9 and 11 throw half away).

Now try • Which searches could be used on each list? (a) `[7, 2, 9, 4, 1]` (b) `[2, 5, 8, 13, 21]`

Check • (a) is not sorted, so only linear search works. (b) is sorted, so both work.

Stretch Watch binary search work

Type this into the IDE. The `print` on line 6 shows each middle item as it is checked.

```
1  def binary_search(items, target):
2      start = 0
3      end = len(items) - 1
4      while start <= end:
5          middle = (start + end) // 2
6          print("checking", items[middle])
7          if items[middle] > target:
8              end = middle - 1
9          elif items[middle] < target:
10             start = middle + 1
11         else:
12             return True
13     return False
14
15 numbers = [3, 8, 12, 15, 21, 26, 30, 34, 41, 47, 52]
16 print(binary_search(numbers, 34))
17 print(binary_search(numbers, 40))
```

Check • The first search prints `checking` 26, 41, 30, 34 and then `True`: the same middle items as your table in Section 3. The search for 40 checks the same four, then prints `False`. Try 52 and 3 next and predict the middle items before you run it.

ANSWER

52: checking 26, 41, 47, 52, True. 3: checking 26, 12, 3, True. Python's `//` is ERL's `DIV`.

The exam answers, one line each

Asked for	Write
Pre-requisite of binary search	The data must be sorted.
How linear search works	Check each item in turn, from the first, until the item is found or the end of the list is reached.
How binary search works	Compare the middle item with the target, discard the half that cannot contain it, and repeat until it is found or no items are left.
An advantage of binary search	Fewer comparisons on a long list, so it is faster.
An advantage of linear search	It works on unsorted data, and it is simpler to code.